Art Tube Design Document

Summary

ArtTube is a security-focused web application for viewing artwork, events, and submitting artwork. Django is used for HTTP Requests, and MySQL is used to store app data.

Architecture

Browser → Django handles URL → views.py returns template → User sends/fetches data from DB → Return updated template

1. Browser accepts a URL.
2. URL is used to select the appropriate function from `views.py`.
3. Users can securely read/write data to/from the database.
4. Updated browser template is generated and displayed to the end user.

Modules

- Models - `artTubeApp/models.py`
- Views - `artTubeApp/views.py`
- Forms - `artTubeApp/forms.py`
- Tests - `artTubeApp/tests.py`
- URLs - `art_tube/urls.py`
- Settings - `art_tube/settings.py`
- Logging - `artTubeApp/log.py`
- Data sanitization - `artTubeApp/sanitize.py`

Tech Stack

- Django, Python, JS, HTML, CSS, MySQL

API

- Python-magic, Pillow, MySQL database, Gmail SMTP, IPware

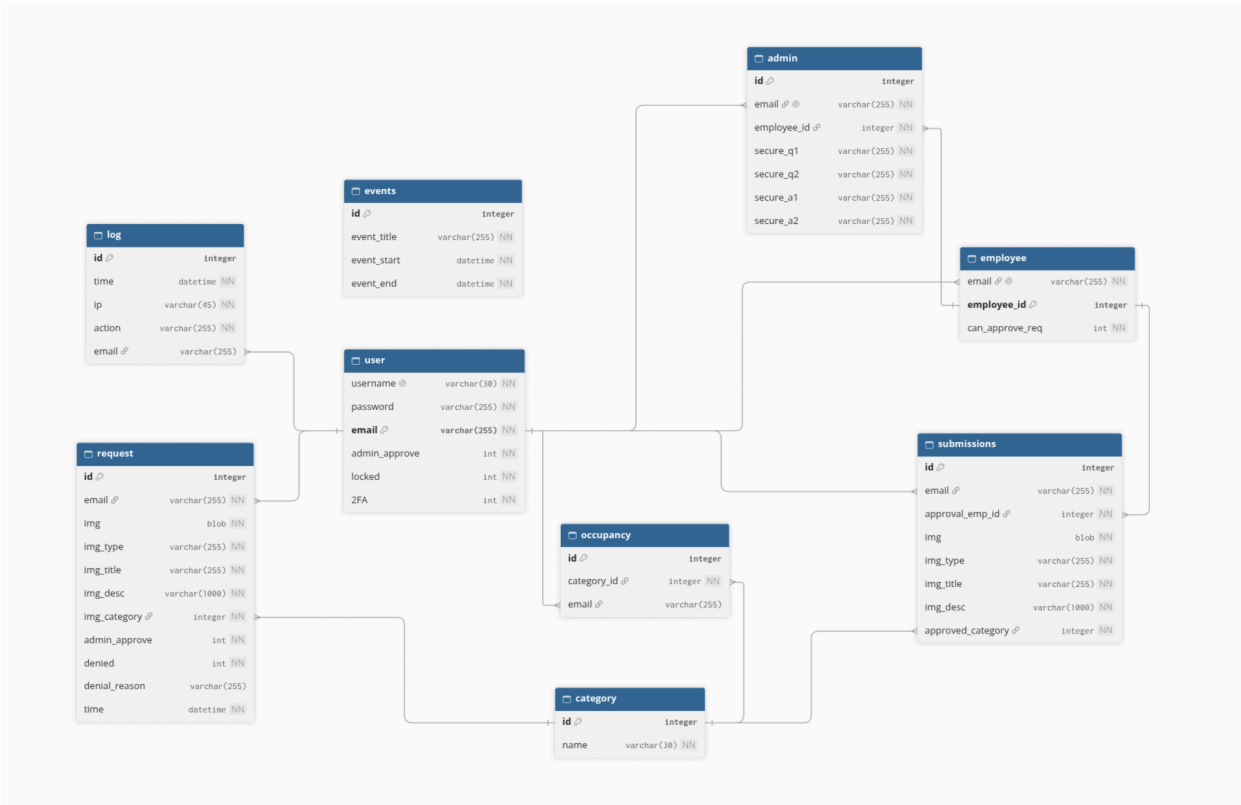
Authentication Mechanisms

- Passwords
- MFA via email codes
- Role based access (admins and employees have higher privilege than users)
- Sessions storing user email - important since user email is used to determine their level of privilege (Admin, Employee, or User)
- Image processing and sanitization
- Password hashing

Data Design

Database includes tables for the following:

- Admin - most privilege
- Employee - less privilege, can approve artwork requests
- User - less privilege, can submit artwork.
- Guest - least privilege, can view art galleries and public events
- Category - categories for artworks (e.g. classical, modern, etc.)
- Events - For storing new events at the gallery
- Log - To store user actions
- Occupancy - Room occupancy
- Request - Store a user request to submit an artwork for review
- Submissions - Store artworks and descriptions



Interface Design

- Security and authentication methods
    - Django includes protection from SQL injection and XSS out of the box
    - Session authentication
    - Password hashing and authentication
    - Authentication via forms
    - Server-side validation
    - CSRF Protection - included in Django
    - ORM database access - reduce risk of injection
    - Role based Authorization
- Message Formats and Data Structures
    - Forms - login, signup, submitting artwork
    - Input types - email, password, image files, username, date/time
    - Database tables - Admin, User, Employee, submissions, etc.

Component Design

Module Details

Models - `artTubeApp/models.py` defines database tables. `python3 manage.py makemigrations` generates the migration files. `python3 manage.py migrate` creates the tables in the database.

Views - `artTubeApp/views.py` defines functions for each URL, and returns a template. Functions may accept user-input from the webpage to create new database objects. Functions may also enable the user to query the database to retrieve objects to display on the browser.

Forms - `artTubeApp/forms.py` defines forms to be used in html. Important for account creation, account validation, submitting artwork requests, and adding events.
Included in files: `login.html, signup.html, emp_dash.html`

URLs - `art_tube/urls.py` defines the URL paths and their associated functions in views.py.

Settings - `art_tube/settings.py` includes connection to database, middleware, URL config, Installed Apps, Password Validation, etc.

Tests `artTubeApp/tests.py` defines tests, to prove application security and protection against attacks. Additionally, tests for functionality to improve web apps and remove errors.

User Interface Design

- Makes use of bootstrap CSS/JS
    - Nav buttons
    - Cards
    - Tables
    - Alerts
    - Gallery Carousel
    - Grids
    - Style

- Navbar - includes the Creator Canvas, Employee Dashboard, Admin Portal, Gallery, and Events
    - Creator Canvas
    - Employee Dashboard - (employees only) Can create new events, and see event history
    - Admin Portal
    - Gallery - Viewing artwork
    - Events - allows users to see events

Threat Model

Interaction points where threats may arise:

- Employees adding events (could add fake events),
- Admin managing roles(can result modify privilege levels of any user),
- User submitting artworks files containing malicious code (Pillow should provide protection),
- User accessing employee/admin dashboard, or Employee accessing admin dashboard
- IP spoofing for the logging system
- Logins - brute force attacks
- Spamming emails to for authentication codes
- SQL injection
- Attacker exploiting another users session identifier

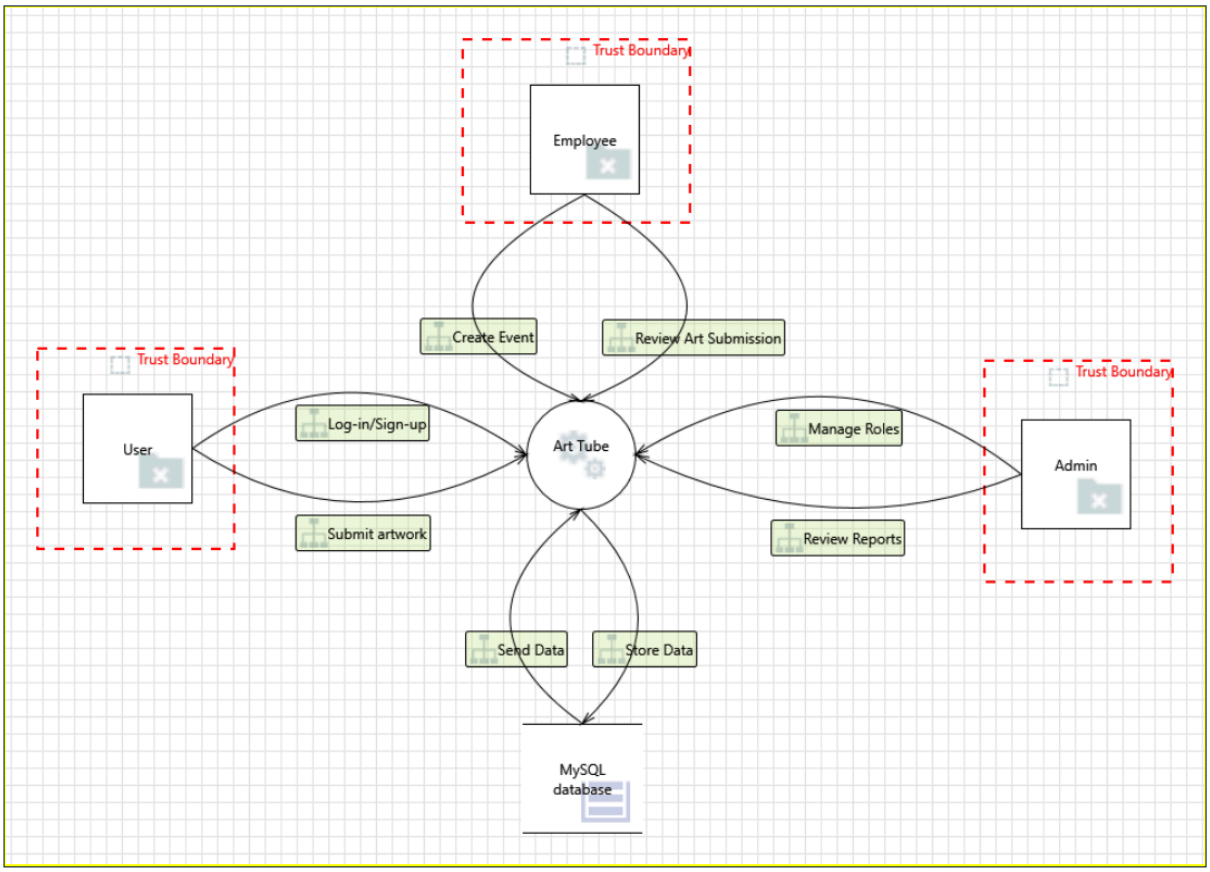
Stride Model

5. Identify Threats (Using STRIDE Model):

Threat Model Summary:

Not Started	144
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	0
Total	144
Total Migrated	0

Diagram: Diagram 1



Potential threats using this framework (over 140 found):

- An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High] Category: Information Disclosure
- An adversary may spoof Employee and gain access to Web Application [State: Not Started] [Priority: High] Category: Spoofing
- An adversary can gain access to sensitive data by sniffing traffic to Web Application [State: Not Started] [Priority: High]

Potential Threats and Priority

- The following includes threats that may or may not need to be solved.

Threat	Priority
Attacker can exploit email as session identifier	High (need random & unpredictable session ID)
Brute force password	High (need minimum password requirements)
SQL injection	Low (django has built in protection)
Unauthorized access to pages or sensitive data	High
Sensitive information in error messages	Low (only when debug = true)
adversary can get access to a user's session due to improper logout and timeout	High
Improper validation / acting on another user’s behalf	High
adversary can get access to a user's session due to insecure coding practices	High
adversary can gain access to certain pages or the site as a whole	Medium

adversary can perform action on behalf of other user due to lack of controls against cross domain requests	High
adversary can steal sensitive data like user credentials	High